

Build a LogiTrack System using Java & Angular:

Server part (Spring Boot)

File 1: Configurations.java:

Method:

public PasswordEncoder passwordEncoder():

- passwordEncoder method need to return BCryptPasswordEncoder.

public WebMvcConfigurer corsConfigurer()

- It should return WebMvcConfigurerAdapter that include method addCorsMappings() it accept CorsRegistry as an argument that allow "GET", "POST", "PUT", "DELETE" operation.

File 2: SecurityConfig.java

Class that extends WebSecurityConfigurerAdapter

Variables:

```
private final UserDetailsService userDetailsService;  
private final JwtRequestFilter jwtRequestFilter;  
private final PasswordEncoder passwordEncoder;
```

Method

public SecurityConfig()

- Inside this constructor argument create instance **UserDetailsService**, **JwtRequestFilter**, **PasswordEncoder** and assign local variable to global variable.

protected void configure(AuthenticationManagerBuilder auth) :

- ‘auth.userDetailsService(userDetailsService).passwordEncoder(passwordEncoder);’
configures the authentication manager to use the provided ‘**UserDetailsService**’ for user details
and the ‘**PasswordEncoder**’ for encoding passwords.

protected void configure(HttpSecurity http) throws Exception:

POST:

- `"/api/register", "/api/login"` which permit all.
- `"/api/business/cargo" , "/api/business/assign-cargo"` permit who has Authority **"BUSINESS"**

GET:

- `"/api/business/drivers" , "/api/business/cargo"` permit who has Authority **"BUSINESS"**
- `"/api/driver/cargo"` permit who has Authority **"DRIVER"**.
- `"/api/customer/cargo-status"` permit who has Authority **"CUSTOMER"**.

PUT:

- `"/api/customer/cargo-status"` Permit who has Authority **"CUSTOMER"**.

public AuthenticationManager authenticationManagerBean() throws Exception

- It should return `authenticationManagerBean()`.

File 3: BusinessController.java

Dependency Injection:

```
private CargoService cargoService;  
private DriverService driverService;
```

Endpoints:

@PostMapping("/api/business/cargo")

- It accept **Cargo** as an **RequestBody** argument perform Insertion and return **ResponseEntity** type **"Cargo"** along with **HTTP STATUS** as **OK**.

@GetMapping("/api/business/drivers")

- This end point need to return all **driver** details.
- Return type should be list of **"Driver"** wrap it with **ResponseEntity**.

@GetMapping("/api/business/cargo")

- This end point need to return all **cargo** details.

- Return type should be list of “Cargo” wrap it with **ResponseEntity**.

@PostMapping("/api/business/assign-cargo")

- It accept two **RequestParam** “cargoId” and “driverId” as an argument.
- If it assigned successfully return in json format “key” is “message” and “Value” is “Cargo assigned successfully”.http status as “ok”
- If not return “Failed to assign cargo” and **HttpStatus** “INTERNAL_SERVER_ERROR”.

File 4: CustomerController.java

- End point for this controller **@RequestMapping("/api/customer")**

Dependency Injection:

```
private CustomerService customerService;
```

End point:

@GetMapping("/cargo-status")

- It accept one **RequestParam** “cargoId” as an argument and return cargo details based on “cargoId”.
- If “cargoId” not present send **HttpStatus** “NOT_FOUND”.

File 5 : DriverController.java

- End point for this controller **@RequestMapping("/api/driver")**

Dependency Injection:

```
private DriverService driverService;
```

End Point:

@GetMapping("/cargo")

- It accept one RequestParam “**driverId**” as an argument and return list of cargo details assigned to the particular driver based on “**driverId**” wrap it using **ResponseEntity**.

@PutMapping("/update-cargo-status")

- It accept two argument “**cargoId**” and “**newStatus**”.
- And update status for the particular **cargoId**.
- If it updated successfully return **ResponseEntity** body message in json format “**key**” is “**message**” and “**Value**” is “**Cargo status updated successfully.**”. http status as “**ok**”.
- If not return “**Failed to update cargo status.**” and **HttpStatus** “**INTERNAL_SERVER_ERROR**”.

File 6: RegisterAndLoginController.java

Dependencies:

```
private UserService userService;
private BusinessService businessService;
private CustomerService customerService;
private DriverService driverService;
private AuthenticationManager authenticationManager;
private JwtUtil jwtUtil;
```

End point:

@PostMapping("/register")

- It accept @RequestBody as User.
- This end point used to register the user.
- Register user based on Roles.

@PostMapping("/login")

- It accept @RequestBody as LoginRequest.
- It return login response as **token, Username, email, role**.

File 7: CargoStatusResponse.java

Variable:

```
private Long cargoId;
private String status;
```

Method:

- Create constructor with argument and getter,setter for the variable.

File 8: LoginRequest.java

Variable:

```
private String username;  
private String password;
```

Method:

- Create constructor with argument and getter,setter for the variable.
 - Constructor should annotate with “**@JsonCreator**”
 - Constructor argument should annotate with **@JsonProperty**.
-

File 9: LoginResponse.java

Variable:

```
private String token;  
private String username;  
private String email;  
private String role;
```

Method:

- Create constructor with argument and getter,setter for the variable.
- Constructor should annotate with “**@JsonCreator**”
- Constructor argument should annotate with **@JsonProperty** for all variables.

File 10: Business.java

Variable:

```
private Long id; //Primary Key and Auto Increment  
private String name;  
private String email;  
@OneToMany(mappedBy = "business")  
@JsonIgnore  
private List<Cargo> cargos;
```

Method:

- Create getter,setter for the variable.

File 11: Cargo.java

Variable:

```
private Long id; //Primary Key and Auto Increment
private String content;
private String size;
private String status; // Status can be 'PENDING', 'IN_TRANSIT', 'DELIVERED'
@ManyToOne(fetch = FetchType.EAGER)
@JoinColumn(name = "business_id")
private Business business;
@ManyToOne(fetch = FetchType.EAGER)
@JoinColumn(name = "driver_id")
private Driver driver;
```

Method:

- Create getter,setter for the variable.

File 12: Customer.java

Variable:

```
private Long id; //Primary Key and Auto Increment
private String name;
private String email;
```

Method:

- Create getter,setter for the variable.

File 13: Driver.java

Variable:

```
private Long id; //Primary Key and Auto Increment
private String name;
private String email;
// Other driver-related properties
@OneToMany(mappedBy = "driver")
@JsonIgnore
private List<Cargo> assignedCargos;
```

Method:

- Create getter,setter for the variable.

File 14: User.java

Variable:

```
private Long id; //Primary Key and Auto Increment
private String username;
private String password;
private String role; // Role can be 'BUSINESS', 'DRIVER', 'CUSTOMER'
private String email ;
```

Method:

- Create getter,setter for the variable.

File 15 : JwtRequestFilter.java

- Implement JWT Filter concept.

File 16 : JwtUtil.java

- Implement JWT Util concept.

File 17 : BusinessRepository.java

Extends JpaRepository for Business

File 18 : CargoRepository.java

Extends JpaRepository for Cargo

Methods:

findByBusinessId -Return List of Cargo

findByDriverId -Return List of Cargo

File 19 : CustomerRepository.java

Extends JpaRepository for Customer

File 20 : DriverRepository.java

Extends JpaRepository for Driver

File 21 : UserRepository.java

Extends JpaRepository for User

Method:

- findByUsername(String username) -Return User.

File 22 : BusinessService.java

- Write a business logic for Business Controller and its relevant Repositories.

File 23 : CargoService.java

Dependencies:

private CargoRepository cargoRepository;
private DriverRepository driverRepository;

Method:

public Cargo addCargo(Cargo cargo)

Write a logic add the cargo.

public List<Cargo> viewAllCargos()

It is get all cargo

public boolean assignCargoToDriver(Long cargoId, Long driverId)

It accept two argument cargoId and driverId.

Find cargo information on its relevant Repository.

Find Driver Information on its relevant Repository.

File 24 : CustomerService.java

Dependencies:

```
private CargoRepository cargoRepository;  
private CustomerRepository customerRepository;
```

Method:

public Customer createCustomer(Customer customer)

Write a method add Customer

public CargoStatusResponse viewCargoStatus(Long cargoId)

Write a method to view cargo status

File 25 : DriverService.java

Dependencies:

```
private DriverRepository driverRepository;  
private CargoRepository cargoRepository;
```

Method

public Driver createDriver(Driver driver)

write a method to add driver

public List<Driver> getAllDrivers()

write a method to get all driver details.

public List<Cargo> viewDriverCargos(Long driverId)

write a method to cargo status assigned to particular driver.

public boolean updateCargoStatus(Long cargoId, String newStatus)

write a method to update cargo status.

File 26 : UserService.java

- This class implements UserDetailsService.

Dependencies

```
private UserRepository userRepository;  
private PasswordEncoder passwordEncoder;
```

Method

public User registerUser(User user)

Write a method to add new user

public User getUserByUsername(String username)

Write a get user details by username.

public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException

Write a method find User by Username .